

DocuMind Enterprise: Sistema Inteligente de Gestión y Análisis Documental

Fase 03: Documento de Desarrollo / Documento técnico

Juan Esteban Rojas Melendez

Universitaria Tecnológica de Santander

## **TABLA DE CONTENIDOS**

1. Descripción del Entorno de Desarrollo
2. Configuración del Proyecto y Gestión de Variables de Entorno
3. Estructura y Organización del Código Fuente
4. Implementación de Base de Datos y Almacenamiento Híbrido
5. Implementación del Backend y Servicios REST
6. Implementación del Pipeline de Procesamiento con Inteligencia Artificial
7. Implementación del Motor de Búsqueda Semántica y RAG
8. Implementación del Frontend y Experiencia de Usuario (UI/UX)
9. Gestión de Errores, Validaciones y Trazabilidad de Estados
10. Control de Versiones y Bitácora de Desarrollo
11. Manual Técnico de Instalación y Ejecución Local

## 1. Descripción del Entorno de Desarrollo

Para dar cumplimiento a los requerimientos de reproducibilidad (RNF-07), se estableció un entorno de desarrollo desacoplado y multiplataforma:

- **Sistema Operativo Host:** Windows 11 Pro (64 bits).
- **Entorno de Ejecución Backend:** Python 3.10+ administrado mediante un entorno virtual (venv) para el aislamiento estricto de dependencias.
- **Entorno de Ejecución Frontend:** Node.js (v20+ LTS) con el gestor de paquetes npm (v10+).
- **Gestor de Base de Datos y Vectores:** PostgreSQL 16 con la extensión pgvector incorporada mediante el contenedor estandarizado pgvector/pgvector:pg16.
- **Motor de Contenedores:** Docker Desktop con Docker Compose v2.
- **Entorno de Desarrollo Integrado (IDE):** Visual Studio Code, configurado con herramientas de soporte estático como Pylance, ESLint, Tailwind CSS IntelliSense y cliente de bases de datos.
- **Proveedor de Inteligencia Artificial:** Google AI Studio API.

## 2. Configuración del Proyecto y Gestión de Variables de Entorno

En cumplimiento con el requerimiento de seguridad RNF-01, las claves secretas y los accesos de red no se encuentran incrustados en el código. La configuración reside en el archivo backend/.env, el cual es ignorado en el repositorio Git:

Fragmento de código

```
# Conexión Relacional y Vectorial
DATABASE_URL=postgresql://postgres:admin@localhost:5432/documind
```

```
# Proveedor de Inteligencia Artificial (Google AI Studio)
GEMINI_API_KEY=AIzaSy...tu_clave_real_de_google_ai_studio
GEMINI_TEXT_MODEL=gemini-3.6-flash
GEMINI_EMBEDDING_MODEL=models/gemini-embedding-2

# Parámetros de Seguridad de Sesión (JWT)
JWT_SECRET_KEY=documind_secure_token_secret_key_uts_2026
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=120
```

### 3. Estructura y Organización del Código Fuente

El proyecto adopta un patrón arquitectónico Cliente-Servidor desacoplado. La estructura física del repositorio se organiza de la siguiente manera:

```
Plaintext
DocuMind-Final/
|
|— docker-compose.yml          # Declaración del contenedor
PostgreSQL + pgvector
|— .gitignore                  # Exclusiones de Git (venv,
node_modules, .env, storage)
|
|— backend/                    # API REST construida sobre
FastAPI
|   |— .env                    # Variables de configuración
local
|   |— requirements.txt        # Dependencias del
ecosistema Python
|   |— main.py                 # Controladores REST,
middleware y arranque
|   |— create_tables.py        # Script DDL para
inicialización de esquemas
|   |— generate_mock_data.py   # Script de generación
masiva del corpus de 30 documentos
|   |— storage/                # Persistencia local de
archivos fuente (.pdf, .docx, .txt)
```

```

|   └─ app/
|       ├── __init__.py
|       ├── database.py           # Instanciación de
SQLAlchemy Engine y sesión get_db
|       └─ models.py             # Modelos ORM relacionales y
definición de campos vectoriales
|
└─ frontend/                     # Aplicación SPA en React +
Vite
    ├── index.html               # Plantilla HTML base
    ├── package.json             # Librerías cliente (axios,
lucide-react, react-router)
    ├── tailwind.config.js       # Configuración del motor de
estilos Tailwind CSS
    ├── vite.config.js           # Empaquetador y
configuración del servidor de desarrollo
    └─ src/
        ├── main.jsx             # Punto de montaje del árbol
DOM
        ├── index.css            # Directivas globales de
estilos
        └─ App.jsx               # Componente unificado:
Dashboard, Repositorio, RAG y Detalle

```

#### 4. Implementación de Base de Datos y Almacenamiento Híbrido

La persistencia combina el modelo relacional tradicional con almacenamiento vectorial denso.

##### Contenedor Docker (docker-compose.yml)

Garantiza la disponibilidad inmediata del servicio con volumen persistente (documind\_postgres\_data):

```

YAML
version: '3.8'
services:

```

```

db:
  image: pgvector/pgvector:pg16
  container_name: documind_db
  restart: always
  environment:
    - POSTGRES_USER=postgres
    - POSTGRES_PASSWORD=admin
    - POSTGRES_DB=documind
  ports:
    - "5432:5432"
  volumes:
    - documind_postgres_data:/var/lib/postgresql/data

volumes:
  documind_postgres_data:

```

### Definición de Esquemas ORM (backend/app/models.py)

Se implementan las entidades Document y DocumentChunk, vinculadas con eliminación en cascada para preservar la integridad referencial (RN-06, RF-07):

- **Tabla documents:** Gestiona el ciclo de vida del archivo mediante las columnas id, filename, file\_path, mime\_type, file\_size\_bytes, status ("Pendiente", "Procesando", "Procesado", "Error"), category, summary, extracted\_data (campo JSONB estructurado), error\_log y created\_at.
- **Tabla document\_chunks:** Administra los fragmentos textuales para el motor RAG. Contiene id, document\_id (FK en cascada), chunk\_index, chunk\_text y embedding Vector(768).

### 5. Implementación del Backend y Servicios REST

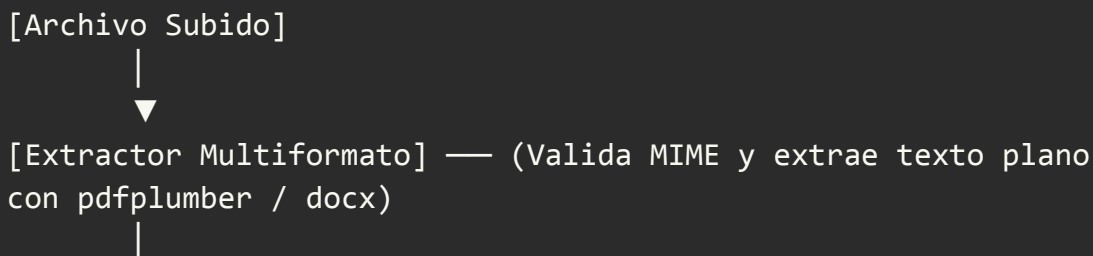
La capa de aplicación fue implementada en FastAPI. Expone las rutas requeridas para cumplir con la matriz de casos de uso:

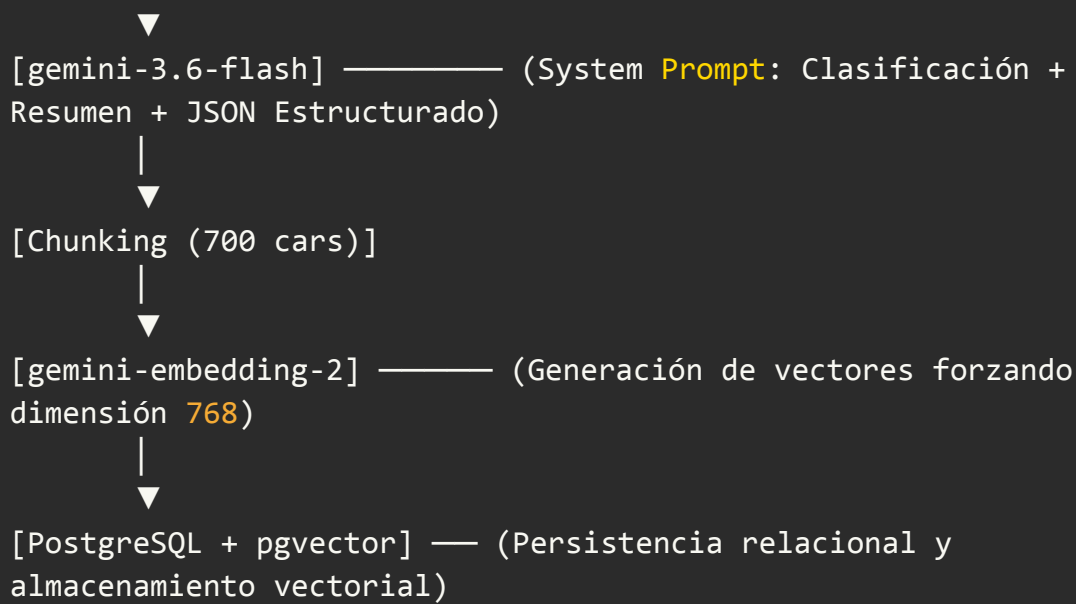
- POST /api/auth/login: Validación de credenciales y generación del token JWT (RF-01).
- GET /api/dashboard/stats: Retorna el conteo global, procesados, fallidos y distribución por tipologías (RF-17).
- POST /api/documents/upload: Admisión multiformato, persistencia en disco e invocación del procesamiento asíncrono con IA (RF-05, RF-08).
- GET /api/documents: Listado general con soporte para filtros por nombre, estado y categoría (RF-15).
- GET /api/documents/{id}: Consulta unitaria del registro, incluyendo metadatos, resumen y JSON extraído.
- GET /api/documents/{id}/download: Transmisión del archivo binario original hacia el navegador (RF-06).
- DELETE /api/documents/{id}: Eliminación física en el sistema de archivos y supresión sincronizada en la base de datos (RF-07).
- POST /api/rag/query: Endpoint de inferencia semántica y preguntas en lenguaje natural (RF-16).

## 6. Implementación del Pipeline de Procesamiento con Inteligencia Artificial

El procesamiento cumple con el ciclo exigido en el requisito central del proyecto:

*archivo → extracción → IA → almacenamiento.*





1. **Extracción Multiformato (RF-08):** Se utiliza pdfplumber para la lectura de documentos .pdf, python-docx para archivos .docx, y lectura con decodificación controlada UTF-8 para archivos .txt.
2. **Clasificación, Resumen y Extracción JSON (RF-09 al RF-13):** El texto extraído se remite al modelo gemini-3.6-flash forzando la salida en formato JSON estricto (response\_mime\_type="application/json"). La IA clasifica en una de las tres tipologías requeridas:
  - *Facturas*: Extrae comprobante, emisor, cliente, fechas y valor total.
  - *Contratos*: Extrae tipología contractual, partes involucradas, objeto, vigencia y valor pactado.
  - *Hojas de Vida*: Extrae candidato, rol principal, años de experiencia, competencias técnicas y máximo nivel educativo.

## 7. Implementación del Motor de Búsqueda Semántica y RAG

Para cumplir con los requisitos RF-14 y RF-16:

1. **Fragmentación (Chunking):** El texto extraído se segmenta en ventanas de 700 caracteres de longitud para mantener la coherencia semántica local.
2. **Vectorización Densa:** Cada fragmento se envía al modelo `models/gemini-embedding-2`. Se pasa de forma explícita el parámetro `output_dimensionality=768` para garantizar la compatibilidad matemática con la columna `Vector(768)` de `pgvector`.
3. **Recuperación por Distancia Coseno:** Cuando el usuario envía una pregunta a `/api/rag/query`, la consulta se vectoriza de manera idéntica y se recuperan los 3 fragmentos más cercanos mediante el operador `<=>` de `SQLAlchemy/pgvector`:
- 4.

```
Python
results = db.query(DocumentChunk.chunk_text, Document.filename,
DocumentChunk.embedding.cosine_distance(q_emb).label("dist"))\
    .join(Document).order_by("dist").limit(3).all()
```

5. **Generación Aumentada (Prompt Augmentation):** Los fragmentos recuperados se concatenan en un bloque de contexto. El LLM sintetiza la respuesta final ciñéndose estrictamente a las fuentes halladas e indicando el nombre de los documentos de soporte.

## 8. Implementación del Frontend y Experiencia de Usuario (UI/UX)

La capa visual se implementó como una SPA reactiva con React y Tailwind CSS, estructurada en cuatro módulos principales:

- **Módulo Dashboard:** Tarjetas visuales de métricas (Total Documentos, Procesados, Errores) e indicadores de distribución por categorías (RF-17).

- **Módulo Repositorio:** Tabla interactiva con selector de archivos que restringe la subida únicamente a .pdf, .docx y .txt (RF-05). Incorpora botones de acción para borrado físico y apertura del detalle.
- **Modal de Inspección de Metadatos y Descarga:** Vista en pantalla dividida que permite inspeccionar el resumen ejecutivo, la estructura JSON formateada y un botón de descarga directa del binario (RF-06).
- **Módulo Asistente IA (Chat RAG):** Consola de chat interactiva con soporte para scroll automático, renderizado de mensajes diferenciados y badges visuales para las fuentes documentales citadas.

## 9. Gestión de Errores, Validaciones y Trazabilidad de Estados

Con el fin de satisfacer las reglas de negocio RN-05 y el requisito RF-18:

- **Secuencia de Estados:** Cada archivo se crea en estado Pendiente, transiciona a Procesando al iniciar la canalización de IA, y finaliza en Procesado si no ocurren incidentes.
- **Resiliencia del Servidor:** Toda la lógica de extracción de texto y consumo de API está contenida dentro de bloques try-except. Si un documento está corrupto o la API de Google alcanza su límite de cuota (Rate Limit), el servidor no se detiene; marca el documento con estado Error, almacena la traza en la columna error\_log y continúa atendiendo peticiones.

## 10. Control de Versiones y Bitácora de Desarrollo

El proyecto utilizó Git para el versionamiento distribuido, aplicando una política de commits atómicos:

| <b>Commit Hash</b> | <b>Componente</b> | <b>Descripción del Cambio Técnico</b>                                                   |
|--------------------|-------------------|-----------------------------------------------------------------------------------------|
| a1c09e1            | Infraestructura   | Configuración de Docker<br>Compose con imagen<br>pgvector/pgvector:pg16 y volúmenes.    |
| b4e72f3            | Base de Datos     | Definición de modelos<br>SQLAlchemy (Document,<br>DocumentChunk) y scripts DDL.         |
| c8a14d5            | Backend           | Creación del extractor<br>multiformato con pdfplumber y<br>python-docx.                 |
| d2f50b8            | Integración IA    | Implementación de prompts<br>estructurados JSON y cálculo de<br>embeddings vectoriales. |
| e9c33a2            | Motor RAG         | Implementación del endpoint<br>/api/rag/query con búsqueda por<br>similitud de coseno.  |
| f5b81c7            | Frontend          | Maquetación responsiva con<br>Tailwind CSS: Dashboard, Repositorio y<br>Chat.           |

|         |               |                                                                                    |
|---------|---------------|------------------------------------------------------------------------------------|
| a4d92e6 | Mantenimiento | Parametrización dinámica de<br>modelos a gemini-3.6-flash y<br>gemini-embedding-2. |
|---------|---------------|------------------------------------------------------------------------------------|

## 11. Manual Técnico de Instalación y Ejecución Local

Guía paso a paso para desplegar y verificar el sistema desde cero en un entorno local (RNF-07):

### Prerrequisitos del Sistema

- Docker Desktop en ejecución.
- Python 3.10 o superior.
- Node.js 18 o superior.
- API Key válida de Google AI Studio.

### Paso 1: Clonación del Repositorio y Despliegue de Base de Datos

```
Bash
git clone https://github.com/usuario/DocuMind-Enterprise.git
cd DocuMind-Enterprise
docker compose up -d
docker exec -it documind_db psql -U postgres -d documind -c
"CREATE EXTENSION IF NOT EXISTS vector;"
```

### Paso 2: Instalación y Arranque del Backend

```
Bash
cd backend
python -m venv venv

# En sistemas Windows:
.\venv\Scripts\activate
# En sistemas Linux/macOS:
```

```
# source venv/bin/activate

pip install -r requirements.txt
python create_tables.py
uvicorn main:app --reload --port 8000
```

### Paso 3: Generación del Corpus de Prueba (30 Documentos)

En una segunda terminal con el entorno virtual activo:

```
Bash
cd backend
python generate_mock_data.py
```

*Este script generará 10 Facturas (.txt), 10 Contratos (.docx) y 10 Hojas de Vida (.pdf) en la carpeta backend/storage/.*

### Paso 4: Instalación y Arranque del Frontend

En una tercera terminal:

```
Bash
cd frontend
npm install
npm run dev
```

### Paso 5: Verificación

- Abrir el navegador web en <http://localhost:5173>.
- Ingresar al Repositorio y cargar los documentos de prueba.
- Verificar en la consola de Swagger (<http://localhost:8000/docs>) la disponibilidad de todos los endpoints REST expuestos.

